SOUGATA SAHA

IT-A2-037 OOP ASSIGNMENT 3

ASSIGNMENT 3

28) 28.Write empty class declarations for the following class hierarchy. Base-> Derived

```
#include<iostream>
using namespace std;
class B{
	public:
	B(){
		cout<<"base ctor"<<endl;
	}
};
class C:public B{
	public:
	C(){
		cout<<"child ctor"<<endl;
	}
};
int main(){
	C ob;
	return 0;
}
```

29) 29.Write empty class declarations for the following class hierarchy. Base1 Base2 Derived

```
#include<iostream>
using namespace std;
class b1{
	public:
	b1(){
		cout<<"base ctor"<<endl;
	}
};
class b2{
	public:
	b2(){
		cout<<"base ctor2"<<endl;
	}
};
```

```
class c:public b1,public b2{
	public:
	c(){
		cout<<"child ctor"<<endl;
	}
};
int main(){
	c ob;
	return 0;
}
```

30) 30.Write empty class declarations for the following class hierarchy. A B C D E F

```
#include<iostream>
using namespace std;
class a{
	public:
	a(){
		cout<<"a"<<endl;
	}
};
class b{
	public:
	b(){
		cout<<" b"<<endl;
	}
};
class c{
	public:
	c(){
		cout<<"c"<<endl;
	}
};
class d:public a,public b{
	public:
	d(){
		cout<<"d"<<endl;
	}
};
class e:public b,public c{
	public:
	e(){
		cout<<"e"<<endl;
	}
```

```
};
class f:public d,public e{
	public:
	f(){

		cout<<"f"<<endl;
	}
};
int main(){
	f ob;
	return 0;
}
```

31) Write a class Person having data member name, age, height etc. Write proper constructors, methods to get/set them and a method printDetails() that prints all information of a person. Now write another class Student from Person and add data members roll, year of admission etc. Write constructors, methods to get/set them and a override printDetails(). Now create a Person and a Student object and call printDetails() function on them to display their information. Now Create an array of pointers to Person and store addresses of two Persons and two Students. Call printDetails() on all elements (a loop may be used). Are you getting output which is supposed to come? Make printDetails() function virtual in the base class and check the result.

```
#include <iostream>
#include <string>
using namespace std;

class Person {
public:
   string name;
   int age;
   float height;

   Person(string n, int a, float h) : name(n), age(a), height(h) {}

   virtual void printDetails() {
      cout << "Name: " << name << ", Age: " << age << ", Height: " << height << endl;
   }
};

class Student : public Person {
public:
   int roll, year;

   Student(string n, int a, float h, int r, int y) : Person(n, a, h), roll(r), year(y) {}
```

```
    void printDetails() {
        cout << "Name: " << name << ", Age: " << age << ", Height: " << height
             << ", Roll: " << roll << ", Year: " << year << endl;
    }
};

int main() {
    Person p("John", 30, 5.9);
    Student s("Mike", 20, 5.7, 123, 2023);

    Person* people[4];
    people[0] = new Person("Alice", 25, 5.6);
    people[1] = new Person("Bob", 35, 5.8);
    people[2] = new Student("Charlie", 22, 5.5, 456, 2021);
    people[3] = new Student("David", 24, 5.9, 789, 2020);

    for (int i = 0; i < 4; i++) {
        people[i]->printDetails();
    }

    return 0;
}
```

32) Write a class Employee having data member name, salary etc. Write proper constructors, methods to get/set them and a virtual method printDetails() that prints all information of a person. Now write two classes Manager and Clerk from Employee. Add 'type' and 'allowance' in the manager and Clerk respectively. Write constructors, methods to get/set them and a override printDetails(). Now create a Manager and a Clerk object and call printDetails() function on them to display their information. Now Create an array of pointers to Employee and store addresses of two Employee, two Managers and two Clerks. Call printDetails() on all elements (a loop may be used). Also find the total salary drawn by all employees.

```
#include <iostream>
#include <string>
using namespace std;

class Employee {
public:
    string name;
    float salary;
```

```
    Employee(string n, float s) : name(n), salary(s) {}

    virtual void printDetails() {
        cout << "Name: " << name << ", Salary: " << salary << endl;
    }
};

class Manager : public Employee {
public:
    string type;

    Manager(string n, float s, string t) : Employee(n, s), type(t) {}

    void printDetails() {
        cout << "Name: " << name << ", Salary: " << salary << ", Type: " << type << endl;
    }
};

class Clerk : public Employee {
public:
    float allowance;

    Clerk(string n, float s, float a) : Employee(n, s), allowance(a) {}

    void printDetails() {
        cout << "Name: " << name << ", Salary: " << salary << ", Allowance: " << allowance << endl;
    }
};

int main() {
    Employee* employees[6];
    employees[0] = new Employee("Alice", 50000);
    employees[1] = new Employee("Bob", 55000);
    employees[2] = new Manager("Charlie", 70000, "Operations");
    employees[3] = new Manager("David", 80000, "Finance");
    employees[4] = new Clerk("Eve", 30000, 5000);
    employees[5] = new Clerk("Frank", 32000, 4500);

    float totalSalary = 0;

    for (int i = 0; i < 6; i++) {
        employees[i]->printDetails();
        totalSalary += employees[i]->salary;
```

```
    }

    cout << "Total Salary: " << totalSalary << endl;

    return 0;
}
```

33) .Write class definitions for the following class hierarchy Shape2D Circle Rectangle The Shape2D class represents two dimensional shapes that should have pure virtual functions area(), perimeter() etc. Implement these functions in Circle and Rectangle. Also write proper constructor(s) and other functions you think appropriate in the Circle and Rectangle class. Now create an array of 5 Shape2D pointers. Create 3 Circle and 2 Rectangles objects and place their addresses in that array. Use a loop to print area and perimeter of all shapes on this array

```
#include <iostream>
#include <cmath>
using namespace std;

class Shape2D {
public:
    virtual float area() = 0;
    virtual float perimeter() = 0;
    virtual ~Shape2D() {}
};

class Circle : public Shape2D {
    float radius;

public:
    Circle(float r) : radius(r) {}

    float area() {
        return M_PI * radius * radius;
    }

    float perimeter() {
        return 2 * M_PI * radius;
    }
};

class Rectangle : public Shape2D {
```

```
    float length, width;

public:
    Rectangle(float l, float w) : length(l), width(w) {}

    float area() {
        return length * width;
    }

    float perimeter() {
        return 2 * (length + width);
    }
};

int main() {
    Shape2D* shapes[5];
    shapes[0] = new Circle(3);
    shapes[1] = new Circle(5);
    shapes[2] = new Circle(7);
    shapes[3] = new Rectangle(4, 5);
    shapes[4] = new Rectangle(6, 8);

    for (int i = 0; i < 5; i++) {
        cout << "Shape " << i + 1 << " -> Area: " << shapes[i]->area()
             << ", Perimeter: " << shapes[i]->perimeter() << endl;
    }

    return 0;
}
```

34) Implement the Shape hierarchy as shown in the figure. Each TwoDShape should contain function getArea to calculate the area of two-dimensional shape. Each ThreeDShape should have member functions getArea and getVolume to calculate the surface area and volume of the three-dimensional shape respectively. Create a program that uses Vector of Shape pointers to objects of each concrete class in the hierarchy. Now write a program that processes all the shapes in the Vector such that if the shape is a TwoDShape it prints name of shape and its area while it prints name of shape, its area and volume if the shape is a ThreeDShape. 35.Write a program to illustrate the role of virtual destructor. Shape TwoDShape ThreeDShape Circle Triangle Ellipse Sphere Cu

```
#include <iostream>
#include <vector>
```

```
#include <cmath>
using namespace std;

class Shape {
public:
   virtual ~Shape() {}
   virtual string getName() = 0;
   virtual float getArea() = 0;
   virtual float getVolume() { return 0; }
};

class TwoDShape : public Shape {
public:
   string getName() {
      return "TwoDShape";
   }
};

class ThreeDShape : public Shape {
public:
   string getName() {
      return "ThreeDShape";
   }
};

class Ellipse : public TwoDShape {
   float a, b;

public:
   Ellipse(float major, float minor) : a(major), b(minor) {}

   float getArea() {
      return M_PI * a * b;
   }
};

class Triangle : public TwoDShape {
   float base, height;

public:
   Triangle(float b, float h) : base(b), height(h) {}

   float getArea() {
      return 0.5 * base * height;
```

```
    }
};

class Circle : public TwoDShape {
    float radius;

public:
    Circle(float r) : radius(r) {}

    float getArea() {
        return M_PI * radius * radius;
    }
};

class Cube : public ThreeDShape {
    float side;

public:
    Cube(float s) : side(s) {}

    float getArea() {
        return 6 * side * side;
    }

    float getVolume() {
        return side * side * side;
    }
};

class Sphere : public ThreeDShape {
    float radius;

public:
    Sphere(float r) : radius(r) {}

    float getArea() {
        return 4 * M_PI * radius * radius;
    }

    float getVolume() {
        return (4.0 / 3.0) * M_PI * radius * radius * radius;
    }
};
```

```
int main() {
    vector<Shape*> shapes;

    shapes.push_back(new Ellipse(4, 2));
    shapes.push_back(new Triangle(3, 6));
    shapes.push_back(new Circle(5));
    shapes.push_back(new Cube(3));
    shapes.push_back(new Sphere(4));

    for (Shape* shape : shapes) {
        cout << "Shape: " << shape->getName() << ", Area: " << shape->getArea();
        if (shape->getVolume() > 0)
            cout << ", Volume: " << shape->getVolume();
        cout << endl;
    }

    return 0;
}
```

35) Write a program to illustrate the role of virtual destructor

```
#include <iostream>
using namespace std;

class Base {
public:
    Base() {
        cout << "Base Constructor" << endl;
    }

    virtual ~Base() {
        cout << "Base Destructor" << endl;
    }
};

class Derived : public Base {
public:
    Derived() {
        cout << "Derived Constructor" << endl;
    }

    ~Derived() {
```

```
        cout << "Derived Destructor" << endl;
    }
};

int main() {
    Base* ptr = new Derived();
    delete ptr;

    return 0;
}
```